



# DSA

## Module 7

## Strings

**Author**  
**Srinivas Dande**



*My* **JAVA**  
**LEARNING**  
**CENTER**



## 7. Strings

- ◆ String is a Sequence of Characters.
- ◆ String can be represented in java in the following ways
  - 1) String class
  - 2) StringBuffer
  - 3) StringBuilder
  - 4) Array of Characters
  - 5) ArrayList of Characters

### 7.1. String Class

- ◆ String is a built-in class in java.lang package.
- ◆ String is final class, so you can't define the subclass for String class.
- ◆ String class implements the following interfaces:
  - ◆ java.io.Serializable
  - ◆ java.lang.Comparable
  - ◆ java.lang.CharSequence
- ◆ String class has following variable to hold data.

```
private final char value[];
```
- ◆ String objects are immutable objects. It means once the object is created then the content or data of the object can't be modified.
- ◆ When you try to modify the contents of object then new object will be created as a result.
- ◆ You can create the object of String in two ways.
  - ◆ Without using new operator.
  - ◆ Using new operator.

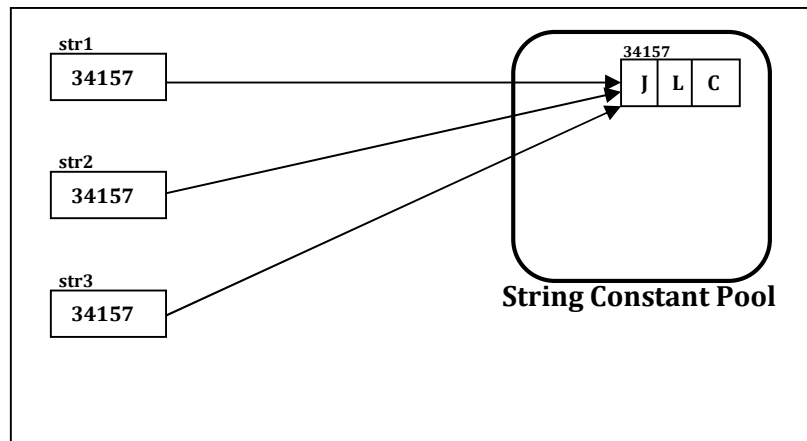


## 7.1.1. Creating String object without new operator

- 1) JVM allocates the memory for the String reference variable.
- 2) JVM verifies the String literal in the String Constant Pool.
- 3) If String literal is not available in the String Constant Pool then it creates new String object inside the String Constant pool and newly created String object address will be assigned to String reference variable.
- 4) If String literal is available then existing String object address will be assigned to String reference variable.

### Demo1.java

```
class Lab457{  
    public static void main(String as[]){  
        String str1="JLC";  
        String str2="JLC";  
        String str3="JLC";  
        System.out.println(str1==str2);  
        System.out.println(str1==str3);  
        System.out.println(str2==str3);  
    }  
}
```

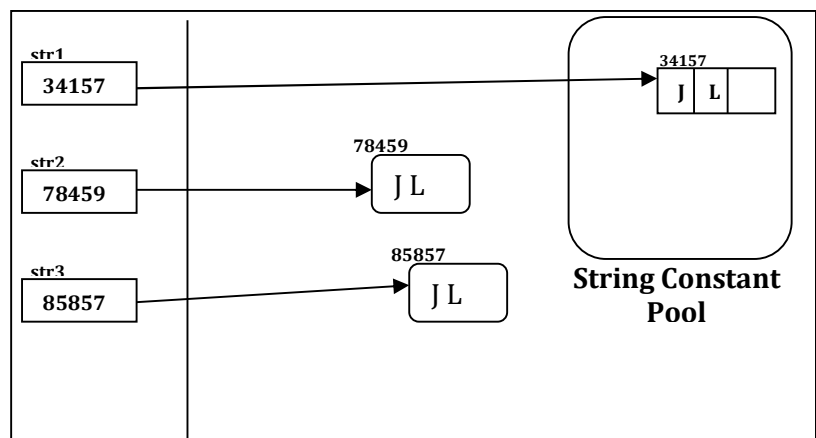


## 7.1.2 Creating String object with new operator

- 1) JVM allocates the memory for the String reference variable.
- 2) JVM verifies the String literal in the String Constant Pool.
- 3) If String literal is not available in the String Constant Pool then it creates new String object inside the String Constant pool.
- 4) If String literal is available in the String Constant Pool then ignores that.
- 5) It creates another new String object outside the constant pool and assigns address of the newly created String object outside the pool to String reference variable.

### Demo2.java

```
class Lab458{  
    public static void main(..... as[]){  
        String str1="JLC";  
        String str2=new String("JLC");  
        String str3=new String("JLC");  
        System.out.println(str1==str2);  
        System.out.println(str1==str3);  
        System.out.println(str2==str3);  
    }  
}
```





## 7.2. StringBuffer Class

- ◆ StringBuffer is a final class in java.lang package.
- ◆ It is used to store the sequence of characters.
- ◆ This is used when there is a necessity to make a lot of modifications to characters of String.
- ◆ StringBuffer is a mutable sequence of characters, It means the contents of the StringBuffer can be modified after creation.
- ◆ Every StringBuffer object has a capacity associated with it.
- ◆ The capacity of the StringBuffer is the number of characters it can hold.
- ◆ The capacity increases automatically as more contents added to it.
- ◆ The methods available in the StringBuffer class are synchronized.
- ◆ StringBuffer class is thread-safe i.e. multiple threads cannot access it simultaneously.

## 7.3. StringBuilder Class

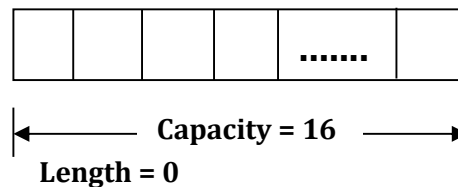
- ◆ StringBuilder is a final class in java.lang package.
- ◆ StringBuilder is a newly added class from Java 5.
- ◆ StringBuilder class functionality is same as StringBuffer only except the methods available in the StringBuilder class are non synchronized.
- ◆ StringBuilder class is not thread-safe i.e. multiple threads can access it simultaneously.



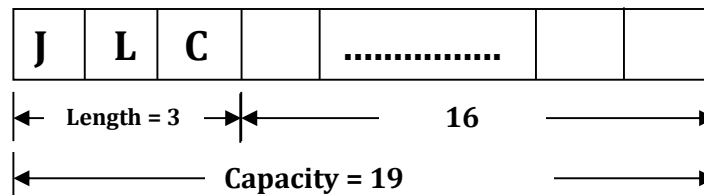
## 7.4. About StringBuffer and StringBuilder

- ◆ equals() is not overridden in StringBuilder and StringBuffer class.
- ◆ When you want to compare contents of StringBuilder or StringBuffer object then you need to convert the object of StringBuilder or StringBuffer into String.
- ◆ hashCode() is not overridden in the StringBuilder and StringBuffer class. It uses the default implementation of Object class.
- ◆ StringBuilder and StringBuffer are not implementing java.lang.Comparable interface, So You can't use compareTo() with StringBuilder and StringBuffer.

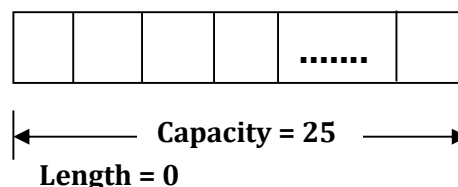
**StringBuilder sb=new StringBuilder();**



**StringBuilder sb=new StringBuilder("JLC");**



**StringBuilder sb=new StringBuilder(25);**





## 7.5. Examples on Strings

### Lab1.java

```
package com.jlcindia.strings;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab1 {
    public static void main(String[] args) {

        String str1="JLC";
        String str2="JLC";

        System.out.println(str1==str2);

        String str3=new String("JLC");
        System.out.println(str1==str3);

    }
}
```



## Lab2.java

```
package com.jlcindia.strings;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab2 {
    public static void main(String[] args) {

        String str1="Hello";
        String str2=" Guys";

        String str3="Hello Guys";
        String str4= str1.concat(str2);
        String str5="Hello".concat(" Guys");

        System.out.println(str3==str4); //F
        System.out.println(str3==str5); //F
        System.out.println(str4==str5); //F

        String str6=str1+str2;
        String str7="Hello" + " Guys"; //IMP- Literal + Literal

        System.out.println(str3==str6);
        System.out.println(str3==str7);

    }
}
```



## Lab3.java

```
package com.jlcindia.strings;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab3 {
    public static void main(String[] args) {

        String str1="JLCD";
        String str2="JLCD";
        String str3="JLCd";

        System.out.println(str1.equals(str2));

        System.out.println(str3.equals(str2));
        System.out.println(str3.equalsIgnoreCase(str2));

        System.out.println(str1.compareTo(str2)); //Same
        System.out.println(str2.compareTo(str3)); //str2 is smaller
        System.out.println(str3.compareTo(str2)); //str3 is Bigger

    }
}
```



## Lab4.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab4 {
    public static void main(String[] args) {

        String str="srinivasd";

        System.out.println(str);
        System.out.println(str.length());

        char chArr[] = str.toCharArray();

        for(int i=0;i<chArr.length;i++)
            System.out.print(chArr[i]+" ");

        System.out.println("\n");

        chArr[0]='S';
        chArr[8]='D';

        for(int i=0;i<chArr.length;i++)
            System.out.print(chArr[i]+" ");

        System.out.println("\n");

        String str1=new String(chArr);
        System.out.println(str1);
        System.out.println(str);
    }
}
```



## Lab5.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab5 {
    public static void main(String[] args) {

        String str="srinivasd";

        System.out.println(str.length());
        System.out.println(str.charAt(0));
        System.out.println(str.charAt(8));

        System.out.println(str.substring(5));
        System.out.println(str.substring(3,8));
        System.out.println(str.substring(3,9));

        System.out.println(str.indexOf("nivas"));
        System.out.println(str.indexOf("i"));
        System.out.println(str.lastIndexOf("i"));

        System.out.println(str.contains("vas"));
        System.out.println(str.contains("nivas"));
        System.out.println(str.contains("hello"));

    }
}
```



## Lab6.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab6 {
    public static void main(String[] args) {

        StringBuffer sb1=new StringBuffer("JLC");
        StringBuffer sb2=new StringBuffer("JLC");

        System.out.println(sb1==sb2);

        System.out.println(sb1.equals(sb2));

        boolean b1= sb1.toString().contentEquals(sb2.toString());
        boolean b2= sb1.toString().equals(sb2.toString());

        System.out.println("b1 : "+b1);
        System.out.println("b2 : "+b2);

        sb1.append(" Guys");
        System.out.println(sb1);

    }
}
```



## Lab7.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab7 {
    public static void main(String[] args) {

        StringBuilder sb1=new StringBuilder("JLC");
        StringBuilder sb2=new StringBuilder("JLC");

        System.out.println(sb1==sb2);

        System.out.println(sb1.equals(sb2));

        boolean b1= sb1.toString().contentEquals(sb2.toString());
        boolean b2= sb1.toString().equals(sb2.toString());

        System.out.println("b1 : "+b1);
        System.out.println("b2 : "+b2);

        sb1.append(" Guys");
        System.out.println(sb1);

        sb1.reverse();
        System.out.println(sb1);
    }
}
```



## 7.6. Problems on Strings

### Problem1.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem1- Reverse the given String
//LeetCode - 344
public class Problem1 {
    //Time Complexity -  $O(n/2) \Rightarrow O(n)$ 
    //Aux Space Complexity :  $O(1)$ 
    public static void reverseString(char[] charArr) {
        int n = charArr.length;
        int start = 0;
        int end = n-1;

        while(start < end) {
            char temp = charArr[start];
            charArr[start] = charArr[end];
            charArr[end] = temp;

            start++;
            end--;
        }
    }

    public static void main(String[] args) {
        String str = "Srinivas";
        System.out.println(str);
        char[] charArr = str.toCharArray();

        reverseString(charArr);
        String revStr = new String(charArr);
        System.out.println(revStr);
    }
}
```



## **Problem2.java**

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem2 : Valid Palindrome or Not
public class Problem2 {
    // Time Complexity -  $O(n/2) \Rightarrow O(n)$ 
    // Aux Space Complexity :  $O(1)$ 
    public static boolean isPalindrome(String str) {

        int n = str.length();
        int start = 0;
        int end = n - 1;

        while (start < end) {
            if (str.charAt(start) != str.charAt(end)) {
                return false;
            }
            start++;
            end--;
        }
        return true;
    }
    public static void main(String[] args) {
        String str = "abcdba";
        System.out.println(str);

        boolean result = isPalindrome(str);
        System.out.println(result);
    }
}
```



## Problem3.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem3 : Reverse the vowels the String
//LeetCode - 345
public class Problem3 {

    public static boolean isVowel(char ch) {
        boolean b = ch=='a' || ch=='e' || ch=='i' || ch=='o' || ch=='u'
                || ch=='A' || ch=='E' || ch=='I' || ch=='O' || ch=='U';
        return b;
    }

    // Time Complexity : O(n)
    // Aux Space Complexity : O(n)
    public static String reverseVowels(String str) {
        char[] charArr = str.toCharArray();

        int n = charArr.length;

        int start = 0;
        int end = n - 1;

        while (start < end) {

            // Keeps the Start Pointer at Vowel
            while (start < n && !isVowel(charArr[start])) {
                start++;
            }

            // Keeps the End Pointer at Vowel
            while (end >= 0 && !isVowel(charArr[end])) {
                end--;
            }
        }
    }
}
```



```
        if (start < end) {
            char temp = charArr[start];
            charArr[start] = charArr[end];
            charArr[end] = temp;
        }
        start++;
        end--;
    }

    String mystr = new String(charArr);
    return mystr;
}

public static void main(String[] args) {
    String str = "hello";
    System.out.println(str);

    String mystr = reverseVowels(str);
    System.out.println(mystr);
}
}
```



## Problem4.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// P4 - Remove Vowels from a String
//LeetCode - 1119
public class Problem4 {

    static boolean isVowel(char c) {
        return c == 'a' || c == 'i' || c == 'e' || c == 'o' || c == 'u' || c == 'A' || c == 'I' || c == 'E' ||
        c == 'O'
        || c == 'U';
    }

    public static String removeVowels(String str) {

        int n = str.length();
        StringBuilder result = new StringBuilder();

        for (int i = 0; i < n; i++) {
            if (!isVowel(str.charAt(i))) {
                result.append(str.charAt(i));
            }
        }
        return new String(result);
    }

    public static void main(String[] args) {
        String str = "srinivas";
        String resultStr = removeVowels(str);
        System.out.println(resultStr);
    }
}
```



## Problem5.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//P5 - Valid Palindrome - 2
//LeetCode - 125

public class Problem5 {

    public static String trimString(String str) {

        str = str.toLowerCase();

        int n = str.length();
        StringBuilder sb = new StringBuilder();

        for (int i = 0; i < n; i++) {
            char ch = str.charAt(i);
            if (Character.isLetterOrDigit(ch))
                sb.append(ch);
        }

        return sb.toString();
    }

    // Time Complexity - O(n)
    // Aux Space Complexity : O(n)
    public static boolean isPalindrome(String str) {

        str = trimString(str);

        int n = str.length();

        int start = 0;
        int end = n - 1;
```



```
        while (start < end) {
            if (str.charAt(start) != str.charAt(end)) {
                return false;
            }

            start++;
            end--;
        }
        return true;
    }

    public static void main(String[] args) {
        String str = "A man, a plan, a canal: Panama";
        System.out.println(str);

        boolean result = isPalindrome(str);
        System.out.println(result);
    }
}
```



## Problem6.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//P6 - Count the Words of given Sentence

public class Problem6 {

    public static String trimSpaces(String str) {
        int start = 0;
        int end = str.length() - 1;

        while (start <= end && str.charAt(start) == ' ') {
            start++;
        }

        while (start <= end && str.charAt(end) == ' ') {
            end--;
        }

        StringBuilder sb = new StringBuilder();
        while (start <= end) {
            char ch = str.charAt(start);

            if (ch != ' ') {
                sb.append(ch);
            } else if (sb.charAt(sb.length() - 1) != ' ')
                sb.append(ch);

            start++;
        }

        return sb.toString();
    }
}
```



```
// Time Complexity - O(n)
// Aux Space Complexity : O(n)
public static int countWords(String str) {
    str = trimSpaces(str);
    int n = str.length();

    int count = 1;
    for (int i = 0; i < n; i++) {
        if (str.charAt(i) == ' ') {
            count++;
        }
    }

    return count;
}

public static void main(String[] args) {

    String str = " Hello Guys Super OK Guys ";
    System.out.println(str.length());
    int count = countWords(str);
    System.out.println(count);

}
```



## **Problem7.java**

---

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//P7 - Reverse the Words of given Sentence
//LeetCode - 151
public class Problem7 {
    static String trimSpaces(String str) {
        int start = 0;
        int end = str.length() - 1;

        while (start <= end && str.charAt(start) == ' ') {
            start++;
        }

        while (start <= end && str.charAt(end) == ' ') {
            end--;
        }

        StringBuilder sb = new StringBuilder();
        while (start <= end) {
            char ch = str.charAt(start);

            if (ch != ' ') {
                sb.append(ch);
            } else if (sb.charAt(sb.length() - 1) != ' ')
                sb.append(ch);

            start++;
        }
        return sb.toString();
    }
}
```



```
static void reverse(char chArr[], int start, int end) {

    while (start < end) {
        char temp = chArr[start];
        chArr[start] = chArr[end];
        chArr[end] = temp;

        start++;
        end--;
    }
}

public static String reverseWords(String str) {
    str = trimSpaces(str);
    int n = str.length();
    char chArr[] = str.toCharArray();
    int start = 0;
    int end = 0;

    while (end < n) {
        if (chArr[end] == ' ') {
            reverseString(chArr, start, end-1);
            start = end + 1;
        }
        end++;
    }
    reverseString(chArr, start, end-1); //Last Word
    reverseString(chArr, 0, n-1); //Last Word

    return String.valueOf(chArr);
}

public static void main(String[] args) {
    String str = " Hello Guys How are You ";
    System.out.println(str);
    String result = reverseWords(str);
    System.out.println(result);
}
}
```



## Problem8.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem8 : Frequency of Characters in given String
//26 Lower Alphabets
//Counting Technique
public class Problem8 {

    // Time Complexity - O(n)
    // Aux Space Complexity : O(1)
    public static void charFrequency(String str) {

        int count[] = new int[26];

        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            count[ch - 97]++;
        }

        for (int i = 0; i < count.length; i++) {
            if (count[i] > 0) {
                char ch = (char) (i + 97);
                System.out.println(ch + "\t" + count[i]);
            }
        }
    }

    public static void main(String[] args) {
        String str = "srinivas";
        System.out.println(str);
        charFrequency(str);
    }
}
```



## Problem9.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem9 : First Repeating Character in given String
//26 Lower Alphabets
//Counting Technique
public class Problem9 {

    // Time Complexity - O(n)
    // Aux Space Complexity : O(1)
    public static int firstRepeating(String str) {

        int count[] = new int[26];

        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            count[ch - 97]++;
        }

        for (int i = 0; i < count.length; i++) {
            if (count[i] > 1)
                return i;
        }
        return -1;
    }
    public static void main(String[] args) {
        String str = "abcdeeffgg";
        int index = firstRepeating(str);
        System.out.println(index);
    }
}
```



## **Problem10.java**

```
package com.jlccindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem10 : First Unique Character in given String
//26 Lower Alphabets
//LeetCode - 387
public class Problem10 {

    // Time Complexity - O(n)
    // Aux Space Complexity : O(1)
    public static int firstUniqChar(String str) {

        int count[] = new int[26];

        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            count[ch - 97]++;
        }

        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            if (count[ch-97] == 1)
                return i;
        }
        return -1;
    }
    public static void main(String[] args) {
        String str = "srinivasr";
        int index = firstUniqChar(str);
        System.out.println(index);
    }
}
```



## Problem11A.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem11A : Frequency of Characters in given String
//Consider Alphabets ( Lower and Upper ) - 52
public class Problem11A {

    // Time Complexity - O(n)
    // Aux Space Complexity : O(1)
    public static void frequencyCount(String str) {

        int count[] = new int[58];

        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            count[ch - 65]++;
        }

        for (int i = 0; i < count.length; i++) {
            if (count[i] > 0) {
                char ch = (char) (i + 65);
                System.out.println(ch + "\t" + count[i]);
            }
        }
    }

    public static void main(String[] args) {
        String str = "abcABCabcABCabcABC";
        System.out.println(str);
        frequencyCount(str);
    }
}
```



## Problem11B.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem11B : Frequency of Characters in given String
//Consider Digits and Alphabets (Lower and Upper) - 62
public class Problem11B {

    // Time Complexity - O(n)
    // Aux Space Complexity : O(1)
    public static void frequencyCount(String str) {

        int count[] = new int[75];

        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            count[ch - '0']++;
        }

        for (int i = 0; i < count.length; i++) {
            if (count[i] > 0) {
                char ch = (char) (i + '0');
                System.out.println(ch + "\t" + count[i]);
            }
        }
    }

    public static void main(String[] args) {
        String str = "123123abcabcABC";
        System.out.println(str);
        frequencyCount(str);
    }
}
```



## Problem11C.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem11C : Frequency of Characters in given String
//Consider All Special Symbols , Alphanumeric - 95
public class Problem11C {

    // Time Complexity - O(n)
    // Aux Space Complexity : O(1)
    public static void frequencyCount(String str) {

        int count[] = new int[95];

        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            count[ch - 32]++;
        }

        for (int i = 0; i < count.length; i++) {
            if (count[i] > 0) {
                char ch = (char) (i + 32);
                System.out.println(ch + "\t" + count[i]);
            }
        }
    }

    public static void main(String[] args) {
        String str = "aaaccAAABB123124###$$$%%@@@*&_--+==";
        System.out.println(str);
        frequencyCount(str);
    }
}
```

**Problem12.java**

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem12 : Remove Duplicate Letters
//LeetCode - 316 ( Not Submiring Now)
public class Problem12 {
    // Time Complexity - O(n)
    // Aux Space Complexity : O(1)
    public static String removeDuplicateLetters(String str) {

        int n = str.length();
        int count[] = new int[26];

        for (int i = 0; i < n; i++) {
            char ch = str.charAt(i);
            count[ch - 97]++;
        }

        StringBuilder sb= new StringBuilder();
        for (int i = 0; i < count.length; i++) {
            if (count[i] != 0) {
                sb.append((char) (i+97));
            }
        }
        return sb.toString();
    }
    public static void main(String[] args) {
        String str = "cbacdcbc";
        System.out.println(str);
        String result= removeDuplicateLetters(str);
        System.out.println(result);
    }
}
```



## **Problem13.java**

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem13 : Valid Anagram
//LeetCode - 242
public class Problem13 {
    public static boolean isAnagram(String str1, String str2) {

        if(str1.length()!=str2.length()) {
            return false;
        }

        int count[] = new int[26];

        for (int i = 0; i < str1.length(); i++) {
            count[str1.charAt(i) - 97]++; // Inc Count
            count[str2.charAt(i) - 97]--; // Dec Count
        }

        for (int i = 0; i < count.length; i++) {
            if (count[i] != 0) {
                return false;
            }
        }
        return true;
    }
    public static void main(String[] args) {
        String str1 = "sri";
        String str2 = "sir";
        boolean result = isAnagram(str1,str2);
        System.out.println(result);
    }
}
```



## **Problem14.java**

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem14 : Is Subsequence
//LeetCode - 392
public class Problem14 {
    public static boolean isSubsequence(String sub, String main) {
        int m = main.length();
        int n = sub.length();

        if (m < n)
            return false;

        int i = 0;
        int j = 0;

        while (i < m && j < n) {
            if (main.charAt(i) == sub.charAt(j)) {
                j++;
            }
            i++;
        }
        return (n == j);
    }

    public static void main(String[] args) {
        String main = "srinivas";
        String sub = "sriav";
        boolean result = isSubsequence(sub, main);
        System.out.println(result);
    }
}
```



## **Problem15.java**

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem15 : FizzBuzz
//LeetCode - 412
public class Problem15 {
    public static List<String> fizzBuzz(int n) {

        List<String> mylist = new ArrayList<>();

        for (int i = 1; i <= n; i++) {

            boolean divBy3 = i % 3 == 0;
            boolean divBy5 = i % 5 == 0;

            if (divBy3 && divBy5) {
                mylist.add("FizzBuzz");
            } else if (divBy3) {
                mylist.add("Fizz");
            } else if (divBy5) {
                mylist.add("Buzz");
            } else {
                mylist.add(i+"");
            }
        }
        return mylist;
    }

    public static void main(String[] args) {
        List<String> mylist = fizzBuzz(6);
        System.out.println(mylist);
    }
}
```

**Problem15A.java**

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem15 : FizzBuzz
//LeetCode - 412
public class Problem15A {
    public static List<String> fizzBuzz(int n) {

        List<String> mylist = new ArrayList<>();
        for (int i = 1; i <= n; i++) {
            boolean divBy3 = i % 3 == 0;
            boolean divBy5 = i % 5 == 0;

            String currString = "";
            if (divBy3) {
                currString+="Fizz";
            }

            if (divBy5) {
                currString+="Buzz";
            }

            if(currString.isEmpty()){
                currString = String.valueOf(i);
            }
            mylist.add(currString);
        }
        return mylist;
    }
    public static void main(String[] args) {
        List<String> mylist = fizzBuzz(6);
        System.out.println(mylist);
    }
}
```

**Problem16.java**

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem16: FizzBuzzZazz
public class Problem16 {
    public static List<String> fizzBuzzZazz(int n) {

        List<String> mylist = new ArrayList<>();

        for (int i = 1; i <= n; i++) {
            boolean divBy3 = i % 3 == 0;
            boolean divBy5 = i % 5 == 0;
            boolean divBy7 = i % 7 == 0;

            if (divBy3 && divBy5 && divBy7) {
                mylist.add("FizzBuzzZazz");
            } else if (divBy3 && divBy5) {
                mylist.add("FizzBuzz");
            } else if (divBy3 && divBy7) {
                mylist.add("FizzZazz");
            } else if (divBy5 && divBy7) {
                mylist.add("BuzzZazz");
            } else if (divBy3) {
                mylist.add("Fizz");
            } else if (divBy5) {
                mylist.add("Buzz");
            } else if (divBy7) {
                mylist.add("Zazz");
            } else {
                mylist.add(i+"");
            }
        }
        return mylist;
    }
}
```



```
public static void main(String[] args) {  
    List<String> mylist = fizzBuzzZazz(21);  
    System.out.println(mylist);  
}  
}
```

## **Problem16A.java**

```
package com.jlccindia.strings;  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */  
//Problem16 : FizzBuzzZazz  
public class Problem16A {  
    public static List<String> fizzBuzzZazz(int n) {  
        List<String> mylist = new ArrayList<>();  
        for (int i = 1; i <= n; i++) {  
            boolean divBy3 = i % 3 == 0;  
            boolean divBy5 = i % 5 == 0;  
            boolean divBy7 = i % 7 == 0;  
  
            String currString = "";  
            if (divBy3)  
                currString+="Fizz";  
  
            if (divBy5)  
                currString+="Buzz";  
  
            if (divBy7)  
                currString+="Zazz";  
  
            if(currString.isEmpty())  
                currString = String.valueOf(i);  
  
            mylist.add(currString);  
        }  
        return mylist;  
    }  
}
```



```
public static void main(String[] args) {  
    List<String> mylist = fizzBuzzZazz(21);  
    System.out.println(mylist);  
}  
}
```

## **Problem17.java**

```
package com.jlcindia.strings;  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */
```

```
//Problem17 : Roman to Integer  
//LeetCode - 13
```

```
public class Problem17 {  
  
    static Map<String, Integer> mymap = new HashMap<>();  
    static {  
        mymap.put("M", 1000);  
        mymap.put("D", 500);  
        mymap.put("C", 100);  
        mymap.put("L", 50);  
        mymap.put("X", 10);  
        mymap.put("V", 5);  
        mymap.put("I", 1);  
    }  
  
    public static int romanToInt(String str) {  
  
        int n = str.length();  
        int sum = 0;  
        int i = 0;
```



```
        while (i < n) {
            String currSymbol = String.valueOf(str.charAt(i));
            int currValue = mymap.get(currSymbol);

            int nextValue = 0;
            if (i + 1 < n) {
                String nextSymbol = String.valueOf(str.charAt(i + 1));
                nextValue = mymap.get(nextSymbol);
            }

            if (currValue >= nextValue) {
                sum = sum + currValue;
                i = i + 1;
            } else {
                sum = sum + (nextValue - currValue);
                i = i + 2;
            }
        }
        return sum;
    }

    public static void main(String[] args) {

        int result = romanToInt("MCMXCIV");
        System.out.println(result);
    }
}
```



## **Problem18.java**

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem18 : Integer to Roman
//LeetCode - 12
public class Problem18 {

    static final int myvalues[] =
        { 1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1 };
    static final String mysymbols[] =
        { "M", "CM", "D", "CD", "C", "XC", "L", "XL", "X", "IX", "V", "IV", "I" };

    public static String intToRoman(int num) {

        StringBuilder sb = new StringBuilder();

        for (int i = 0; i < myvalues.length; i++) {
            while (num >= myvalues[i]) {
                num = num - myvalues[i];
                sb.append(mysymbols[i]);
            }
        }
        return sb.toString();
    }

    public static void main(String[] args) {
        String result = intToRoman(23);
        System.out.println(result);
    }
}
```